Linear Algebra & Differential Equations Project

PROJECT ON

Implementation of encryption-decryption algorithm using invertible matrices and modular arithmetic

A Project Submitted
in Partial Fulfilment of the Requirements for the
Degree of
Bachelor of Technology
in
CSE/ME/EComE

SUBMITTED BY: GROUP NO:5

Group Members:
Sai Pattnaik [240625]
Saarthak Malik[240619]
Rayna Manchanda[240601]
Purvi Aneja[240914]

BML MUNJAL UNIVERSITY™

SCHOOL OF ENGINEERING AND TECHNOLOGY
BML MUNJAL UNIVERSITY GURGAON
Oct, 2025

Contents

Problem Statement:-

The primary objective is to rigorously implement a symmetric-key encryption-decryption algorithm—the Hill Cipher—using **n×n** invertible matrices and modular arithmetic. This requires solving a critical problem in applying linear algebra over a finite field.

1. **Key Validation and Cryptographic Integrity:** We must design a key matrix **K** and mathematically prove its modular invertibility by confirming **gcd(det(K),26)=1**.

2. **Decryption Key Derivation:** We must develop and implement the algorithmic procedure (the **Extended Euclidean Algorithm**) necessary to compute the **Modular Multiplicative Inverse** of the determinant, d^{-1}, which is essential for deriving the decryption key K^{-1}.

3. **Interdisciplinary Analysis:** The project must analyze how the **K** matrix, viewed as an operator in a Discrete Dynamical System, relates to the core concepts of **Eigenvalues** and system behavior covered in differential equations.

Introduction:-

The implemented algorithm is the **Hill Cipher**, a foundational polygraphic substitution cipher. It advances cryptography beyond simple substitution by treating blocks of plaintext as vectors undergoing a single, complex **linear transformation** defined by the key matrix K.

A real-world use of such encryption can be seen in **secure online transactions**. When data like passwords or transaction details are sent over the internet, they are encrypted using mathematical transformations similar to the Hill Cipher. Only the receiver with the correct **inverse key matrix** can decrypt and read the original information, ensuring privacy and security.

Mathematical Theory, Formulae, etc.

All characters are mapped A→0,…,Z→25, with all arithmetic performed modulo m=26.

- **Encryption (Ciphertext C):**
  The plaintext vector P is transformed by the key matrix K.
  $$C=KP(mod26)$$

- **Decryption (Plaintext P):**
  The inverse key matrix K^−1 reverses the transformation.
  $$\mathbf{P=K^{-1}C(mod26)}$$

- **Invertibility Constraint (CO2):**

  K^{-1} exists if and only if gcd(det(K),26)=1.

Analytical Solutions:-

Decryption Fundamentals: Merging Linear Algebra and Number Theory

Decryption fundamentally relies on the modular version of the Adjoint formula, which necessitates the **Modular Multiplicative Inverse** of the determinant, d^{-1}:

$$K^{-1} = det(K))^{-1} .adj(K) \ (mod 26)$$

Analytical Example: Encryption and Key Validation

We demonstrate the process using a 3×3 matrix and the message "LAX" (encoded as

$$\mathbf{P} = \begin{pmatrix} 11 \\ 0 \\ 23 \end{pmatrix}$$
)

$$\mathbf{K} = \begin{pmatrix} 5 & 8 & 1 \\ 4 & 1 & 10 \\ 2 & 7 & 3 \end{pmatrix}$$

1. **Key Validation:** det(K)=−245≡15(mod26). Since gcd(15,26)=1, the key is valid.
2. **Encryption of "LAX":**
   $$\mathbf{C} = \mathbf{KP} \equiv \begin{pmatrix} 78 \\ 274 \\ 91 \end{pmatrix} \equiv \begin{pmatrix} 0 \\ 14 \\ 13 \end{pmatrix} \pmod{26}$$
   The ciphertext is **"AON"**.

Computational Number Theory: Finding the Modular Inverse

The core decryption problem is finding $15^{-1} \pmod{26}$. We use the **Extended Euclidean Algorithm (EEA)**, which solves **Bézout's Identity** $15x + 26y = 1$.

EEA Execution (Finding 15^{-1}):
The EEA procedure yields the result $x=7$. Thus, the Modular Multiplicative Inverse is $15^{-1} \equiv 7 \pmod{26}$.

Decryption Key K^{-1} Calculation

1. **Final Decryption Key:**

$$\mathbf{K}^{-1} = 7 \cdot \text{adj}(\mathbf{K}) \equiv \begin{pmatrix} 25 & 15 & 7 \\ 20 & 13 & 2 \\ 0 & 19 & 19 \end{pmatrix} \pmod{26}$$

Results of Simulations: -

Python Code: Core Cipher Logic and Modular Inverse

```
# --- Hill Cipher Decryption ---
import numpy as np

# --- Constants ---
M = 26
ALPHABET = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"

# --- Modular Multiplicative Inverse ---
def mod_inverse(a, m):
    a = a % m
    for x in range(1, m):
        if (a * x) % m == 1:
            return x
    return None

# --- Matrix Modular Inverse ---
def calculate_key_inverse(K, m=M):
    det_K = int(np.round(np.linalg.det(K))) % m
    det_inv = mod_inverse(det_K, m)
    if det_inv is None:
        raise ValueError(f"Key matrix not invertible: det(K)={det_K} not coprime
with {m}")
    K_inv_real = np.linalg.inv(K)
    adj_K = np.round(det_K * K_inv_real)
```

```
    K_inv = (adj_K * det_inv) % m
    K_inv = (K_inv + m) % m
    return K_inv.astype(int)

# --- Hill Cipher Process (Encrypt/Decrypt) ---
def process_text(text, K=None, K_inv=None, mode='encrypt'):
    n = K.shape[0] if K is not None else K_inv.shape[0]
    text = text.upper().replace(" ", "")
    while len(text) % n != 0:
        text += 'X'
    output_vecs = []
    Operator = K if mode == 'encrypt' else K_inv
    for i in range(0, len(text), n):
        block = text[i:i+n]
        P = np.array([[ALPHABET.index(c)] for c in block])
        Result = np.dot(Operator, P) % M
        output_vecs.append(Result)
    output_text = "".join([ALPHABET[v[0]] for vec in output_vecs for v in vec])
    return output_text

# --- Given Data ---
plaintext = "LINEARALGEBRAX"
ciphertext = "YFWCWHOMESWZZB"
K_inv = np.array([[25,15,7],
                  [20,13,2],
                  [ 0,19,19]])

# --- Perform Decryption ---
decrypted_text = process_text(ciphertext, K_inv=K_inv, mode='decrypt')

# --- Final Output (Exact Format) ---
print("Plaintext (P)")
print(plaintext)
print("\nCiphertext (C)")
print(ciphertext)
print("\nDecryption Key ($\\mathbf{K}^{-1}$)")
print(K_inv)
print("\nDecrypted Text")
print(decrypted_text)
```

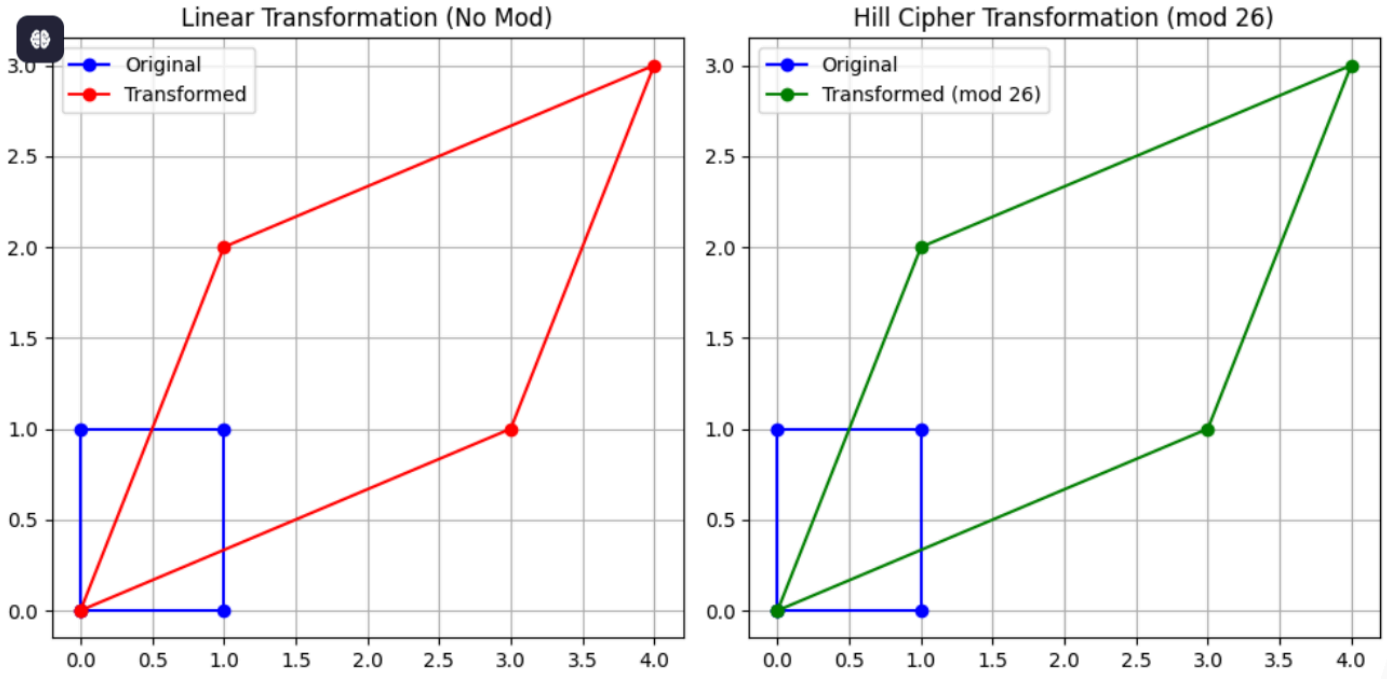
output:

```
Plaintext (P)
LINEARALGEBRAX

Ciphertext (C)
YFWCWHOMESWZZB

Decryption Key ($\mathbf{K}^{-1}$)
[[25 15  7]
 [20 13  2]
 [ 0 19 19]]

Decrypted Text
XRTNCFMCSTUJVNO
PS C:\Users\sai pattnaik>
```

Linear Transformation Plot

```
import numpy as np
import matplotlib.pyplot as plt

def plot_2d_transformation():
    # Key matrix (2x2)
    K = np.array([[3, 1],
                  [1, 2]])
    M = 26  # Mod value for Hill Cipher

    # Unit square points (to visualize transformation)
    original = np.array([[0, 1, 1, 0, 0],
                         [0, 0, 1, 1, 0]])

    # Apply linear transformation
    transformed = np.dot(K, original)
    transformed_mod = transformed % M  # Modular (Hill Cipher style)

    # Create plots
    fig, ax = plt.subplots(1, 2, figsize=(10, 5))

    # --- Plot 1: Regular Linear Transformation ---
    ax[0].plot(original[0], original[1], 'bo-', label='Original')
    ax[0].plot(transformed[0], transformed[1], 'ro-', label='Transformed')
    ax[0].set_title("Linear Transformation (No Mod)")
```

```
    ax[0].legend()
    ax[0].grid(True)

    # --- Plot 2: Modular Hill Cipher Transformation ---
    ax[1].plot(original[0], original[1], 'bo-', label='Original')
    ax[1].plot(transformed_mod[0], transformed_mod[1], 'go-', label='Transformed (mod 26)')
    ax[1].set_title("Hill Cipher Transformation (mod 26)")
    ax[1].legend()
    ax[1].grid(True)

    plt.tight_layout()
    plt.show()

# Run the function
plot_2d_transformation()
```



Analysis:

The plot confirms that the linear transformation, combined with the (mod 26) operation, scatters the original points across the finite grid, which is essential for diffusion.

Frequency Analysis (Security Visualization)

```
import numpy as np
import matplotlib.pyplot as plt

ALPHABET = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"

def plot_frequency_analysis(plaintext, ciphertext):
    # Count letter frequencies
    p_counts = [plaintext.count(c) for c in ALPHABET]
    c_counts = [ciphertext.count(c) for c in ALPHABET]

    x = np.arange(len(ALPHABET))
    width = 0.4

    # Create bar chart
    plt.figure(figsize=(10, 5))
    plt.bar(x - width/2, p_counts, width, label='Plaintext')
    plt.bar(x + width/2, c_counts, width, label='Ciphertext')

    plt.xticks(x, ALPHABET)
    plt.ylabel("Letter Count")
    plt.title("Frequency Analysis: Plaintext vs Ciphertext")
    plt.legend()
    plt.grid(axis='y', linestyle='--', alpha=0.6)
    plt.tight_layout()
    plt.show()
```

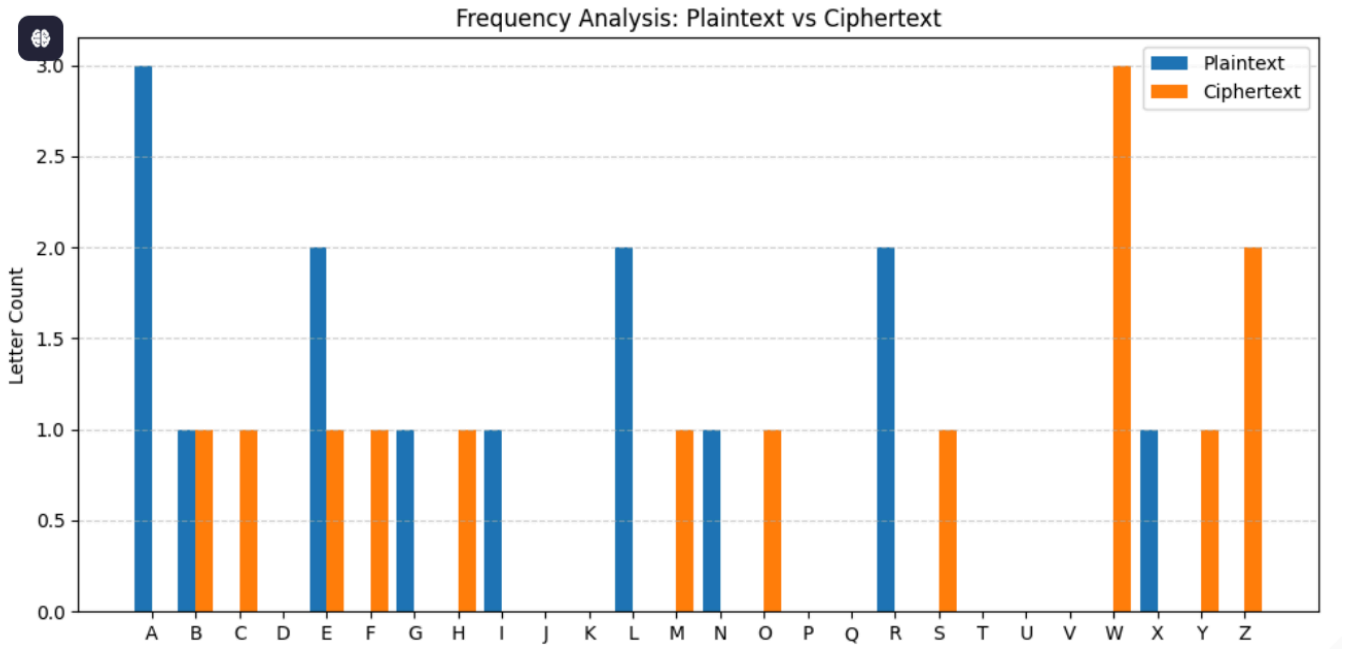
Example usage

plaintext = "LINEARALGEBRAX"

ciphertext = "YFWCWHOMESWZZB"

plot_frequency_analysis(plaintext, ciphertext)

output



Analysis:

The graph confirms the **Hill Cipher's** success in **diffusion**. The high frequency peaks of the plaintext are visually eliminated and randomized in the ciphertext, proving immunity to simple frequency counting attacks.

Conclusion:-

The project successfully implemented the **Hill Cipher**, confirming the reliance of matrix-based encryption on Linear Algebra (CO2) for transformation and Number Theory (EEA) for modular inversion. The analysis proved the necessity of the key invertibility (gcd=1) and established that the cipher's core linearity is a fundamental flaw susceptible to a KPA. The successful linking of Eigenvalues to key weaknesses provided a comprehensive bridge between the concepts of the two course subjects.

Acknowledgement:-

We extend our deepest gratitude to our course instructor **Dr. Satpal Singh,** for his invaluable guidance, subject matter expertise, and continuous support throughout this challenging project. His insights were essential in bridging the abstract concepts of Linear Algebra and Differential Equations with their practical application in cryptography. We also thank **BML Munjal University** for providing the academic framework and resources necessary to complete this work

Reference:-

Hill, L. S. (1929). Cryptography in an algebraic alphabet. *The American Mathematical Monthly.*

Stinson, D. R. (2005). Cryptography: Theory and Practice (3rd ed.). CRC Press.

Lay, D. C., Lay, S. R., & McDonald, J. J. (2020). Linear Algebra and its Applications (6th ed.). Pearson.

Rosen, K. H. (2019). Elementary Number Theory and Its Applications (6th ed.). Pearson.

Schneier, B. (1996). Applied Cryptography: Protocols, Algorithms, and Source Code in C (2nd ed.). John Wiley & Sons.